FIT1058 — Chapter 3: Proofs

Exercise Solutions

Alston

Semester 2, 2026

Exercise 3

Assuming the first of De Morgan's Laws,

$$\neg(P \vee Q) = \neg P \wedge \neg Q,$$

prove the second of De Morgan's Laws,

$$\neg(P \wedge Q) = \neg P \vee \neg Q,$$

as simply as possible.

Exercise 4

Prove the two Distributive Laws ((4.2) and (4.3) in § 4.14).

Exercise 6

Prove that

$$(P_1 \wedge \cdots \wedge P_n) \Rightarrow C \quad \text{is logically equivalent to} \quad \neg P_1 \vee \cdots \vee \neg P_n \vee C.$$

A disjunction of the form $\neg P_1 \vee \cdots \vee \neg P_n \vee C$ is called a *Horn clause*. These play a big role in the theory of logic programming.

Exercise 7

Simplify the following expression as much as possible, using Boolean algebra:

$$(\neg X \wedge \neg Y) \vee (\neg X \wedge Y) \vee (X \wedge Y).$$

Exercise 8

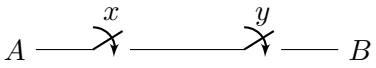
This question is about using Boolean algebra to describe the algebra of switching.

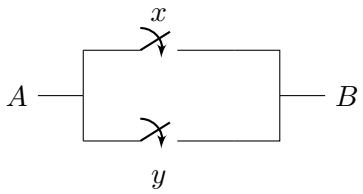
An electrical switch is represented diagrammatically as follows.

The state of a switch can be represented by a Boolean variable, with the states Off and On being represented by *False* and *True* respectively. Let x be a Boolean variable representing the state of a switch. Then $x = \mathit{True}$ represents the switch being On, so that electrical current can pass through it; $x = \mathit{False}$ represents the switch being Off, so that there is no electrical current through the switch.

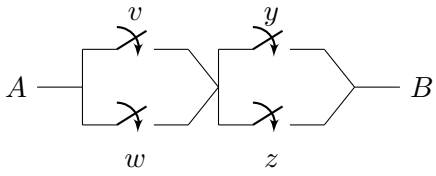
We can put switches together to make more complicated circuits. Those circuits can then be described by Boolean expressions in the variables that represent the switches. In the following circuits, v, w, x, y, z are Boolean variables representing the indicated switches.

(a) For each of the two switching circuits below, write a Boolean expression in x and y for the proposition that current flows between the two ends of the circuit. (The ends are shown as A and B in the diagram. In effect, the diagrams only show part of a complete circuit. The rest of the circuit would contain a power supply, such as a battery, and an electrical device that operates when the current flows, such as a light or an appliance.)

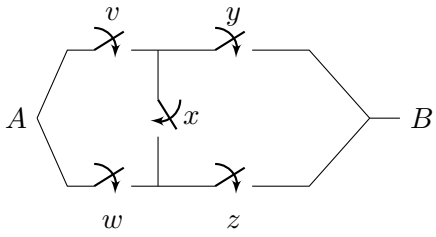




(b) For each of the next two circuits, again construct a Boolean expression to represent the proposition that current flows between the two ends of the circuit. Compare the two expressions you construct. What can you say about the relationship between them?



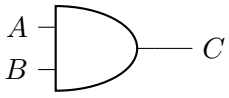
(c) Similarly, construct Boolean expressions for the next two circuits. Compare the two expressions you construct. What can you say about the relationship between them?



(d) For each of our circuit pairs in (a)–(c), discuss how they are related to each other, and what this means for how the Boolean expressions derived from them are related to each other.

Exercise 18

Suppose you have an unlimited supply of AND gates, which are logic gates for the binary operation $\wedge$. Each gate has two inputs, for the two arguments, and one output, which gives the conjunction of the inputs. The output of one gate can be used for the input of another gate.



Show how to put AND gates together to compute the conjunction of eight logical arguments $x_1, x_2, \ldots, x_8$.

How many AND gates do you need?

Suppose each AND gate computes its output as soon as both its inputs are available, and that it takes time t to compute the output. Assume that all the initial inputs $x_1, x_2, \ldots, x_8$ are available simultaneously, at time 0. How long does your combination of AND gates take to compute its output?

Try and put your AND gates together to minimise the total time taken to compute the final output.

Exercise 19

Prove, by induction on n, that the conjunction of 2^n inputs can be computed in time nt by a suitable combination of AND gates.

Hence prove that the conjunction of m arguments (where m is not necessarily a power of 2) can be computed by a suitable combination of AND gates in time $\lceil \log_2 m \rceil$.

Exercise 20

A Boolean expression is called *affine* if it is of one of the following forms:

$$x_1 \oplus x_2 \oplus \cdots \oplus x_n \qquad \text{or} \qquad x_1 \oplus x_2 \oplus \cdots \oplus x_n \oplus \mathit{True}.$$

Prove, by induction on n, that for all $n \in \mathbb{N}$, the number of satisfying truth assignments of an affine Boolean expression with n variables is 2^{n-1}.

It follows that half of all truth assignments are satisfying for the expression, and half are not.

Here, a *truth assignment* is just an assignment of a truth value to each variable, and it is *satisfying* if the assignment makes the whole expression *True*.

Exercise 21

Let x_1x_0 be the bits of a two-bit binary number x, representing an integer in $\{0, 1, 2, 3\}$. So $x_1, x_0 \in \{0, 1\}$ and we have $x = 2x_1 + x_0$.

Let $y_2y_1y_0$ be the three-bit binary representation of the number $x + 1$ obtained from x by incrementing it. So $y \in \{1, 2, 3, 4\}$, and its leading bit is 0 except if $x = 3$, in which case $y = 4$, which in binary is 100.

In this exercise we use 0 and 1 for the truth values *False* and *True* (as mentioned on p. 122 in § 4.1α).

Give Boolean expressions for each of the three bits of y, in terms of the two bits x_1 and x_0 of x.

Exercise 22

Is the operation set $\{\oplus, \neg\}$ universal? Justify your answer.

Exercise 23

Prove or disprove the claim that there exists a universal operation set containing just a single operation.

You need not restrict consideration to the three operations $\wedge, \vee, \neg$. You can use another operation of at most two arguments (in other words, a binary operation) if you wish.

What type of proof did you use? (Recall the proof types discussed in Chapter 3.)

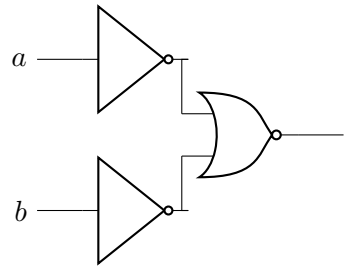
Exercise 24

Computer circuits actually perform the very same Boolean logic that we have studied using *logic gates* (§ 4.4α). The idea is that you have one or more wires coming into a gate, and usually one

output wire. If an input wire has current flowing through, the variable it represents is set to *True*, and if not, then *False*. The logic gate then takes those signals and gives off an output signal if the result should be *True*, and no signal if the result is *False*. Below are the most common logic gates seen in circuits.

The AND, OR, and NOT gates function the same as what we have seen previously. If both signals of an AND gate are on, it will output a signal. For OR, only one input signal is required to be on, and for NOT, there will only be an output signal if the input signal is off (*False*).

XOR is "EXCLUSIVE OR", which outputs a signal only if a or b are on, but not both. NAND, NOR, and XNOR are simply the inversions of AND, OR, and XOR respectively (i.e. NAND is "not both", NOR is "neither", and XNOR is "both or neither"). These gates can also be connected in series, like in the example below, which is equivalent to the logical expression $(\neg a \vee \neg b)$.



(a) What single logic gate is the above circuit equivalent to?

(b) We saw in § 4.20, that $\{\vee, \wedge, \neg\}$ is a universal set of operations, meaning that any logical expression can be expressed with only these operations. How would you express the function of an XOR gate as a Boolean expression using only these operations?

(c) Draw a circuit diagram which performs the same functionality as an XOR gate using only AND, OR, and NOT gates.

(d) Do we even need three types of gates? Given that $\{\wedge, \vee, \neg\}$ is a universal set of operations, use De Morgan's Law to prove that $\{\wedge, \neg\}$ and $\{\vee, \neg\}$ are also universal sets of operations. (Hint: Can you express $\vee$ with the other two operators?)

(e) Draw a circuit diagram that performs an OR operation using only AND and NOT gates, and also one which performs AND using only OR and NOT gates.

(f) Can you perform the function of a NOT gate, or an AND gate, with only NAND gates? What does this tell you about NAND gates?